Programación II

Guía 5

Programación C++ sin POO

Plataformas: Windows 10/11 y GNU/Linux
Herramientas: Visual Studio Code, GCC, G++ y GDB

Edición 2026

Propósito de esta guía

Esta guía cubre todo lo visto desde el repaso de archivos en C hasta el planteo del problema que motiva la Programación Orientada a Objetos: `struct`, `std::cout` / `std::cin`, control de flujo, funciones, `std::string`, `std::vector`, referencias, sobrecarga de funciones y archivos con `fstream`.

Todavía no se utilizan clases; ese contenido se trabajará en la **Guía 6: Programación Orientada a Objetos**.

Cómo está organizada esta guía: los primeros 10 ejercicios son de resolución obligatoria y se trabajarán en clase. Los 20 que siguen, ubicados al final de la guía como **Ejercicios adicionales**, se recomiendan resolverlos para seguir practicando y profundizando los contenidos desarrollados, pero no se van a corregir ni tratar formalmente en clases.

Resultados de aprendizaje

Al finalizar podrás:

- reconocer las principales continuidades y diferencias entre C y C++;
- utilizar `std::cout` y `std::cin` para realizar operaciones de salida y entrada;
- definir y utilizar estructuras mediante `struct` para agrupar datos relacionados;
- utilizar `std::string` para trabajar con cadenas de caracteres;
- utilizar `std::vector` para almacenar y recorrer colecciones de datos;
- utilizar referencias como mecanismo de paso de parámetros;
- distinguir el uso de punteros y referencias en situaciones sencillas;
- definir y utilizar funciones sobrecargadas;
- utilizar `std::ifstream` y `std::ofstream` para leer y escribir archivos de texto;
- compilar y depurar programas en C++ utilizando Visual Studio Code, G++ y GDB;
- reconocer qué conocimientos desarrollados previamente en C continúan siendo aplicables en C++.

Antes de comenzar

En las guías anteriores se trabajó con programas escritos en lenguaje C utilizando Visual Studio Code, GCC y GDB.

A partir de esta guía se incorpora el lenguaje C++. Este cambio **no implica abandonar los conocimientos ni las herramientas utilizadas previamente**. Las estructuras de control, las funciones, los vectores, los punteros, el trabajo con archivos y los mecanismos de compilación y depuración continúan formando parte del trabajo cotidiano.

Los archivos fuente escritos en C continuarán utilizando la extensión `.c`, mientras que los programas escritos en C++ utilizarán la extensión `.cpp`.

Para compilar programas en C se continuará utilizando `gcc`, mientras que para los programas escritos en C++ se utilizará `g++`. El depurador seguirá siendo GDB.

	C	C++
Extensión del archivo fuente	`.c`	`.cpp`
Compilador	`gcc`	`g++`
Estándar utilizado	`C17`	`C++17`
Depurador	GDB	GDB
Compilar en VS Code	`Ctrl+Shift+B`	`Ctrl+Shift+B`
Depurar en VS Code	`F5`	`F5`

La carpeta `.vscode` también evoluciona

La carpeta `.vscode` utilizada hasta ahora no será reemplazada.
Los archivos `tasks.json` y `launch.json` serán **ampliados** para incorporar nuevas tareas de compilación y depuración para C++, manteniendo al mismo tiempo las configuraciones correspondientes a C.
De esta manera, dentro del mismo entorno de trabajo será posible continuar compilando y depurando ejercicios anteriores escritos en C y comenzar a trabajar con los nuevos programas escritos en C++.

Para resolver los ejercicios utiliza una carpeta general denominada, por ejemplo, `Guia05`. Una posible estructura inicial es:

```
ProgramacionII/
`-- Guia05/
    |-- .vscode/
    |   |-- tasks.json
    |   `-- launch.json
    |
    |-- ejercicio01.cpp
    |-- ejercicio02.cpp
    |-- ejercicio03.cpp
    |-- ...
    `-- ejercicio30.cpp
```

La carpeta completa debe abrirse desde Visual Studio Code para reutilizar la configuración de compilación y depuración almacenada dentro de `.vscode`.

Una compilación desde terminal equivalente a la que realizará la tarea de Visual Studio Code puede escribirse, por ejemplo, como:

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -g -O0 ejercicio01.cpp -o ejercicio01
```

Las opciones `-g` y `-O0` mantienen el mismo propósito que en C: permitir que GDB relacione con claridad el código fuente con la ejecución del programa durante una sesión de depuración.

Flujo de trabajo

Para cada ejercicio se recomienda:

1. analizar la consigna antes de comenzar a programar;
2. identificar qué conocimientos de C continúan siendo aplicables;
3. reconocer qué herramienta propia de C++ se incorpora en el ejercicio;
4. crear y guardar el archivo fuente correspondiente con `Ctrl+S`;
5. compilar con `Ctrl+Shift+B`;

6. corregir los errores o advertencias informados por G++;
7. ejecutar el programa desde la terminal integrada;
8. probarlo con distintos datos de entrada;
9. utilizar el depurador mediante `F5` cuando sea necesario analizar variables, referencias, vectores o flujo de ejecución.

Glosario rápido

Los siguientes términos aparecen en varios ejercicios:

- **Canal (WiFi):** las redes WiFi funcionan dividiendo el espectro de radio en "canales" numerados (por ejemplo, del 1 al 11), cada uno correspondiendo a una franja de frecuencia distinta. Es un número entero que identifica en qué franja transmite un punto de acceso.
- **SSID:** el nombre con el que se identifica una red WiFi (el nombre que ves al buscar redes disponibles en tu celular).
- **dBm:** unidad logarítmica para expresar potencia de una señal, tomando como referencia 1 milivatio. Un valor típico de potencia transmitida ronda los 20–46 dBm; un valor típico de señal *recibida* es negativo (por ejemplo, -65 dBm), porque la señal se debilita mucho en el camino.
- **RSSI (Received Signal Strength Indicator):** indica qué tan fuerte llega una señal inalámbrica al receptor, en dBm. Son valores negativos: cuanto más cerca de 0, más fuerte es la señal (-50 dBm es muy buena, -90 dBm es muy débil).
- **RSRP:** el equivalente al RSSI, pero específico de redes celulares (4G/5G).
- **SNR (relación señal/ruido):** compara la potencia de la señal útil contra la del ruido de fondo, en dB. Cuanto mayor el SNR, más "limpia" se puede recibir la señal.
- **Latencia:** el tiempo que tarda un dato en viajar desde el origen hasta el destino, medido en milisegundos (ms). Para aplicaciones interactivas (videollamadas, juegos online), valores bajos son mejores.
- **Jitter:** la variación de la latencia entre una medición y la siguiente. Aunque el promedio de latencia sea bueno, un jitter alto indica una conexión inestable.

Ejercicios

Ejercicio 1: Mediciones de canal WiFi

Definí un `struct Medicion` con los campos `canal` (`int`) y `rssi` (`double`, en dBm — ver glosario). Declará un array de 5 elementos, cargalo con valores fijos en el código (por ejemplo, canales 1, 6, 11, 6 y 1, con RSSI entre -85 y -60), y mostrá todas las mediciones con `std::cout`.

Ejercicio 2: Resumen de un punto de acceso

Escribí un programa que le pida al usuario, con `std::cin`, el SSID (`std::string`), el canal (`int`) y la frecuencia en MHz (`double`) de un punto de acceso WiFi, y muestre un resumen con `std::cout` en una sola línea.

Dato de referencia: las redes WiFi de 2.4 GHz usan frecuencias entre 2400 y 2483 MHz aproximadamente.

Ejercicio 3: Clasificación de señal celular

El RSRP (ver glosario) es, en redes celulares, el indicador que usa el teléfono para saber qué tan bien recibe la señal de la antena más cercana.

Leé un valor de RSRP y clasificá la señal con `if` / `else if` / `else`:

- "Excelente" si es mayor a -80 dBm;
- "Buena" si es mayor a -95 dBm;
- "Débil" si es mayor a -110 dBm;
- "Sin cobertura" en cualquier otro caso.

Estos umbrales son valores de referencia típicos; no hace falta memorizarlos.

Ejercicio 4: Tipo de red y simulación de latencias

Pedí un código de tipo de red:

- 1 = WiFi;
- 2 = 4G;
- 3 = 5G;
- 4 = Ethernet.

Mostrá el nombre correspondiente utilizando `switch`.

Después, usando un `for`, generá y mostrá 8 mediciones simuladas de latencia en milisegundos (ver glosario). Podés usar:

```
rand() % 50 + 5
```

para simular valores entre 5 y 54 ms, un rango razonable para una conexión hogareña.

Ejercicio 5: Conversión de velocidad y tiempo de descarga

Las velocidades de internet se anuncian en **Mbps** (megabits por segundo), pero el tamaño de los archivos se mide en **MB** (megabytes).

La relación es 1 byte = 8 bits, así que para pasar de Mbps a MBps (megabytes por segundo) hay que dividir por 8.

Escribí una función:

```
double mbpsAKBps(double mbps)
```

que convierta Mbps a **kilobytes por segundo** (dividir por 8 y multiplicar por 1000), y otra función:

```
double tiempoDescargaSeg(double tamanioMB, double velocidadMbps)
```

que estime cuántos segundos tarda una descarga (tamaño en MB dividido la velocidad en MBps).

Probá ambas desde `main()`.

Ejercicio 6: Validación simplificada de una IP

Una dirección IP (la identificación numérica de un dispositivo en una red) tiene el formato `A.B.C.D`, con tres puntos separando cuatro números (por ejemplo, `192.168.1.1`).

Escribí una función:

```
bool formatoIpValido(std::string ip)
```

que devuelva `true` solo si la cadena contiene exactamente 3 puntos.

Se trata de una validación simplificada: no hace falta comprobar que cada número esté entre 0 y 255.

Podés usar `find()` o recorrer la cadena con un `for`.

Ejercicio 7: Estadísticas de latencia

Cargá 6 mediciones de latencia (ms, ver glosario) ingresadas por teclado en un `std::vector<double>` y calculá el promedio y el *jitter* (ver glosario) entre cada medición y la siguiente, es decir, la diferencia en valor absoluto.

Ejercicio 8: Rango de SNR con referencias

El SNR (ver glosario) se mide en dB, y cuanto más alto, mejor.

Escribí una función:

```
void minMaxSNR(std::vector<double> valores,
               double &minimo,
               double &maximo)
```

que reciba un vector de mediciones de SNR y devuelva, por referencia, el valor mínimo y el máximo.

Ejercicio 9: Potencia de un amplificador, sobrecargada

La potencia en dBm se calcula a partir de la potencia en vatios mediante:

$$P_{\text{dBm}} = 10 \log_{10} \left(\frac{P_{\text{W}}}{0.001} \right) \text{ (0.001 vatios = 1 milivatio, la referencia del dBm)}$$

Si en cambio se conoce el voltaje y la impedancia de un circuito, la potencia en vatios se obtiene mediante:

$$P = \frac{V^2}{Z}$$

Escribí una función `calcularPotenciaDbm` sobrecargada:

- una versión que reciba la potencia en vatios directamente;
- otra que reciba voltaje e impedancia y calcule primero la potencia en vatios antes de convertirla a dBm.

Aplicalo a un amplificador de señal de un repetidor de red.

Ejercicio 10: Log de latencia con archivos

Escribí un programa que simule 10 mediciones de latencia (ms) de una conexión, las muestre por pantalla y las guarde, una por línea, en un archivo `latencia.txt` utilizando `std::ofstream`.

Después, en el mismo programa, abrí el archivo con `std::ifstream`, calculá el promedio y determiná si alguna medición superó los 100 ms (un valor alto, que marcaríamos como "pico" de latencia).

Lista de comprobación final

Antes de finalizar la guía, comprueba que puedes:

- ☐ compilar un programa escrito en C++ utilizando `g++`;
- ☐ diferenciar un archivo fuente `.c` de un archivo fuente `.cpp`;
- ☐ utilizar `std::cout` y `std::cin`;
- ☐ definir y utilizar un `struct`;
- ☐ utilizar una variable de tipo `std::string`;
- ☐ crear, cargar y recorrer un `std::vector`;
- ☐ enviar datos a una función mediante referencias;
- ☐ reconocer la diferencia básica entre un puntero y una referencia;
- ☐ definir y utilizar funciones sobrecargadas;
- ☐ escribir información en un archivo mediante `std::ofstream`;
- ☐ leer información desde un archivo mediante `std::ifstream`;
- ☐ continuar compilando y depurando programas anteriores escritos en C;
- ☐ compilar y depurar programas en C++ desde Visual Studio Code;
- ☐ utilizar `Ctrl+Shift+B` para compilar y `F5` para depurar;
- ☐ reconocer qué elementos de la programación en C continúan utilizándose en C++.

Ejercicios adicionales

Los siguientes ejercicios permiten continuar practicando y combinar los contenidos desarrollados en la guía.

Ejercicio 11: Tabla de nodos de red

Un puerto es un número (0 a 65535) que identifica un servicio específico dentro de un dispositivo en red (por ejemplo, el puerto 80 suele usarse para tráfico web).

Definí:

```
struct NodoRed
{
    std::string ip;
    int puerto;
    bool activo;
};
```

Declará un array de 5 nodos con datos a elección y escribí una función que cuente cuántos están activos.

Ejercicio 12: Redes de invitados

Muchos routers hogareños permiten crear una segunda red WiFi, separada de la principal, pensada para visitas (“red de invitados”).

Cargá en un `std::vector<std::string>` una lista de SSIDs (nombres de red) ingresados por teclado y contá cuántos contienen la palabra `invitado"` en cualquier parte del nombre, utilizando `find()`.

Ejercicio 13: Reintento con espera exponencial

Cuando un dispositivo no logra conectarse, muchos sistemas usan una técnica llamada *backoff exponencial*: en vez de reintentar todo el tiempo, esperan cada vez más tiempo entre intento e intento.

Simulá este mecanismo: usando un `while`, para cada intento (empezando en 1) calculá el tiempo de espera como 2 elevado al número de intento, en segundos. Podés utilizar `pow(2, intento)`.

Mostrá cada valor y continuá hasta el intento número 5.

$$\text{Intento } 1 \to 2 \text{ s}, \quad \text{Intento } 2 \to 4 \text{ s}, \quad \text{Intento } 3 \to 8 \text{ s}, \quad \ldots$$

Ejercicio 14: Potencias fuera de rango

Escribí una función:

```
void contarFueraDeRango(std::vector<double> potencias,
                        double minimo,
                        double maximo,
                        int &cantidad)
```

que recorra un vector de potencias de transmisión (en dBm) y devuelva, por referencia, cuántas están fuera del rango permitido (por ejemplo, fuera de 0 a 46 dBm, un rango típico para equipos WiFi).

Ejercicio 15: Tasa de éxito de transmisión

En una red, no todos los paquetes de datos enviados llegan correctamente a destino; pueden perderse por interferencia, congestión, etc.

Definí:

```
struct Paquete
{
    std::string origen;
    std::string destino;
    int bytes;
    bool exitoso;
};
```

Cargá un array de 8 paquetes con datos a elección y calculá el porcentaje de paquetes transmitidos con éxito.

Ejercicio 16: Tabla de ruteo persistente

Una **tabla de ruteo** le indica a un dispositivo de red hacia dónde reenviar cada paquete: por cada red de **destino**, guarda el **gateway** (la "puerta de entrada" — el siguiente dispositivo al que hay que enviarle el paquete) y una **métrica** (un número que indica qué tan preferible es esa ruta; menor métrica, mejor ruta).

Definí:

```
struct Ruta
{
    std::string destino;
    std::string gateway;
    int metrica;
};
```

Escribí un programa que guarde un array de 4 rutas en un archivo de texto (una ruta por línea, campos separados por comas), y otro fragmento que las vuelva a leer con `std::ifstream` y las muestre.

Ejercicio 17: Pérdida de trayecto con dos modelos

Cuando una señal viaja de una antena a otra, va perdiendo potencia con la distancia.

El modelo más simple, para espacio libre (sin obstáculos), es:

$$\text{perdida_dB} = 32.44 + 20\log_{10}(\text{distancia_km}) + 20\log_{10}(\text{frecuencia_MHz})$$

En un entorno urbano, con edificios y obstáculos, la señal se pierde más rápido. Un modelo simplificado agrega un término extra:

$$\text{perdida_dB} = \text{perdida_espacio_libre} + 10n\log_{10}(\text{distancia_km})$$

donde n es un "exponente de propagación"; cuanto más obstáculos, mayor n, con valores típicos entre 2 y 4.

Escribí dos versiones sobrecargadas de una función `perdidaTrayecto`:

- una que implemente la primera fórmula;
- otra que reciba además el exponente n e implemente la segunda.

Ejercicio 18: Parseo de un encabezado HTTP

Cuando un navegador pide una página web, el servidor responde con "encabezados" (líneas de texto con información sobre la respuesta), uno de los cuales suele ser `Content-Length`, que indica cuántos bytes tiene el contenido.

Dada una cadena como:

```
Content-Length: 348
```

escribí un programa que extraiga el número después de los dos puntos y lo guarde en un `int`, usando `find()`, `substr()` y `std::stoi()`.

Es el mismo patrón utilizado en clase para el comando AT del módem, aplicado a otro protocolo.

Ejercicio 19: Batería de nodos IoT

Un nodo IoT ("Internet de las Cosas") es un dispositivo pequeño, generalmente a batería, que envía datos por red, por ejemplo un sensor remoto de temperatura.

Cargá en un `std::vector<int>` el nivel de batería (0 a 100 %) de 10 nodos IoT.

Escribí una función que reciba el vector y devuelva, por referencia, un `std::vector<int>` con los índices de los nodos en estado crítico (batería menor a 15 %).

Ejercicio 20: Log de errores de red

Escribí un programa que, cada vez que se ejecuta, agregue al archivo `errores.txt` una línea con un código de error simulado (un entero entre 1 y 5, representando distintos tipos de fallas de red) y un número de "instante" simulado, sin borrar lo anterior (modo `std::ios::app`).

Después, leé el archivo completo y contá cuántas veces apareció cada código de error.

Ejercicio 21: Checksum simple de un paquete

Un **checksum** es un número calculado a partir de los datos de un paquete, que se envía junto con el paquete para que el receptor pueda verificar si los datos llegaron sin errores. El receptor recalcula el checksum con los datos recibidos y lo compara con el que venía adjunto; si no coinciden, algo se corrompió en el camino.

Los checksums reales, como CRC, son más sofisticados. En este ejercicio se utilizará uno simplificado.

Simulá los bytes de un paquete como un `std::vector<int>` (valores entre 0 y 255).

Calculá un checksum simple: la suma de todos los bytes, módulo 256, para que el resultado siempre quede entre 0 y 255.

Ejercicio 22: Selección de estación base

Cuando te movés con el celular, el teléfono constantemente decide a qué antena (estación base) conectarse y elige la que le da mejor señal. Este proceso se llama *handover*.

Definí:

```
struct EstacionCercana
{
    std::string nombre;
    double rssi;
};
```

Cargá un array de 4 estaciones base con distintos valores de RSSI (ver glosario) y escribí una función que determine a cuál conviene conectarse: la de mayor RSSI, es decir, la más cercana a 0.

Ejercicio 23: Analizador de log de accesos

Cargá en un `std::vector<std::string>` líneas con formato:

```
IP - ESTADO
```

por ejemplo:

```
192.168.1.5 - OK
192.168.1.9 - FALLIDO
```

simulando un registro de intentos de acceso a un servidor.

Contá cuántos accesos fueron exitosos y cuántos fallidos, buscando la palabra clave dentro de cada línea.

Ejercicio 24: Distancia máxima de un enlace

Usando la fórmula de pérdida en espacio libre del Ejercicio 17, dado un `std::vector<double>` con varias distancias (en km) y una frecuencia fija (por ejemplo, 2400 MHz), calculá la pérdida para cada distancia.

Utilizando una función con parámetros por referencia, determiná la distancia máxima para la cual el enlace sigue siendo "viable".

En este ejercicio se considerará viable cualquier pérdida menor a 120 dB. Este umbral es arbitrario y se utiliza únicamente para practicar.

Ejercicio 25: Reporte estadístico de una antena

La ganancia de una antena (en dBi) indica cuánto concentra la señal en una dirección respecto a una antena ideal que irradia por igual en todas direcciones. Más ganancia generalmente significa más alcance en esa dirección.

Escribí un programa que lea, desde un archivo `mediciones_antena.txt` (una medición de ganancia por línea), el promedio, el mínimo y el máximo.

El archivo puede generarse mediante un programa auxiliar o cargarse a mano con un editor de texto.

Escribí esos tres valores en un nuevo archivo denominado `reporte_antena.txt`.

Ejercicio 26: Validación de configuración de un router

Definí:

```
struct ConfiguracionRouter
{
    int canal;
    double potencia;
    std::string ssid;
};
```

Escribí una función que reciba una configuración y devuelva, por referencia, un `std::vector<std::string>` con los mensajes de error encontrados:

- canal fuera de 1–13;

- potencia fuera de 0–30 dBm;
- SSID vacío.

El rango indicado para el canal corresponde al rango habitual de canales WiFi en 2.4 GHz y el de potencia se utiliza como referencia para este ejercicio.

Ejercicio 27: Cobertura de antenas, con sobrecarga

El área de cobertura de una antena, si conocemos su radio de alcance, es simplemente el área de un círculo:

$$A = \pi r^2$$

Si en cambio conocemos la potencia de transmisión y la sensibilidad del receptor (la potencia mínima que el receptor necesita para "escuchar" la señal), podemos estimar un radio aproximado a partir del margen entre ambas (potencia transmitida menos sensibilidad, en dB).

Para este ejercicio alcanza con una relación simplificada e inventada, por ejemplo:

$$\text{radio_km} = \frac{\text{margen_dB}}{20}$$

Esta relación se utiliza únicamente para practicar la sobrecarga; no representa un modelo real de propagación.

Escribí dos versiones sobrecargadas de una función `double areaCobertura(...)`:

- una que reciba el radio directamente;
- otra que reciba potencia y sensibilidad y estime primero el radio.

Aplicá ambas a un `std::vector` de antenas.

Ejercicio 28: Red de sensores

Definí:

```
struct Sensor
{
    int id;
    std::string tipo;
    double valor;
};
```

Los tipos posibles serán:

- `"temperatura"`;
- `"humedad"`;
- `"voltaje"`.

Cargá un array de 9 sensores (3 de cada tipo), escribí una función que calcule el promedio por tipo y generá un reporte con los tres promedios en un archivo de texto.

Ejercicio 29: Gestión de ancho de banda

El **ancho de banda** de un enlace es la cantidad máxima de datos que puede transportar por segundo, en Mbps.

Si varios usuarios comparten un mismo enlace y entre todos consumen más de lo que el enlace soporta, hay congestión y, en la práctica, cada usuario termina recibiendo menos velocidad de la que pidió.

Definí:

```
struct Conexion
{
    std::string usuario;
    double consumoMbps;
};
```

Dado un y una capacidad total del enlace (en Mbps), escribí una función que determine si el consumo total supera la capacidad y, si es así, devuelva por referencia el porcentaje en que habría que reducir el consumo de cada usuario para no superarla.

Ejercicio 30: Mini sistema de monitoreo

Combinando lo aprendido en toda la guía, escribí un programa que simule 5 estaciones base.

Cada estación deberá representarse mediante un `struct` que contenga:

- nombre;
- potencia;
- estado de dos sensores asociados, por ejemplo temperatura del equipo y voltaje de alimentación.

El programa deberá:

1. guardar el estado completo de las estaciones en un archivo;
2. generar un reporte separado que contenga únicamente las estaciones que tengan algún valor fuera de rango.

Este ejercicio es, a propósito, muy parecido a lo que se resolverá con clases en la próxima guía.

A medida que lo resuelvas, prestá atención a qué partes del código se sienten repetitivas o difíciles de mantener: esas son, justamente, las que la Programación Orientada a Objetos va a simplificar.